

4 - Strings

Alejandro Veloz

aavelozb.github.io/py



Objetivos de la unidad

- Usar el tipo str para **almacenar y procesar texto**.
- Acceder a los caracteres de un string por su **índice** y extraer **substrings** (rebanadas).
- Comparar strings **lexicográficamente**, entendiendo el rol del **código ASCII**.
- **Recorrer** un string carácter a carácter, con while y con for.
- Resolver problemas que requieran **buscar patrones** dentro de un string y **construir** nuevos strings.

El problema

Hasta ahora los datos eran **un sólo valor**:

```
edad = 20  
promedio = 5.4  
aprobado = True
```

El problema

Hasta ahora los datos eran **un sólo valor**:

```
edad = 20  
promedio = 5.4  
aprobado = True
```

Pero en muchos problemas hay texto:

```
rut = '15136120-K'  
patente = 'CRTJ 32'  
fecha = '01/06/2001'  
tweet = 'Hola #usm'
```

El problema

Hasta ahora los datos eran **un sólo valor**:

```
edad = 20  
promedio = 5.4  
aprobado = True
```

Pero en muchos problemas hay texto:

```
rut = '15136120-K'  
patente = 'CRTJ 32'  
fecha = '01/06/2001'  
tweet = 'Hola #usm'
```

Un string es una secuencia de caracteres con una *estructura interna* que nos interesa **inspeccionar**:

¿Cuál es la última letra?, ¿qué hay antes del guión?, ¿aparece un #?

Strings: lo básico

Strings literales

Un string literal se delimita con comillas **simples** o **dobles**. Ambas formas son equivalentes.

```
mensaje = 'Hola'  
saludo  = "Hola"
```


Strings literales

Un string literal se delimita con comillas **simples** o **dobles**. Ambas formas son equivalentes.

```
mensaje = 'Hola'
saludo  = "Hola"
```

- Un string puede contener **cualquier** carácter: letras, dígitos, espacios, signos.
- '42' es un **string**, no un número: '42' + '1' da '421'.
- El string **vacío** se escribe '' (o "") y tiene largo 0.

Strings literales

Un string literal se delimita con comillas **simples** o **dobles**. Ambas formas son equivalentes.

```
mensaje = 'Hola'
saludo  = "Hola"
```

- Un string puede contener **cualquier** carácter: letras, dígitos, espacios, signos.
- '42' es un **string**, no un número: '42' + '1' da '421'.
- El string **vacío** se escribe '' (o "") y tiene largo 0.

Si el texto contiene una comilla simple, se delimita con dobles.

```
frase = 'Sólo vuela "el que se atreve"'
```

Lectura de strings como entrada

`input()` **siempre** entrega un string. Hasta ahora lo convertíamos:

```
edad = int(input('Edad: '))      # convertimos a int
```

Lectura de strings como entrada

`input()` **siempre** entrega un string. Hasta ahora lo convertíamos:

```
edad = int(input('Edad: '))      # convertimos a int
```

Ahora muchas veces **no** queremos convertir:

```
nombre = input('Cómo te llamas? ') # se queda como str
```

Lectura de strings como entrada

`input()` **siempre** entrega un string. Hasta ahora lo convertíamos:

```
edad = int(input('Edad: '))      # convertimos a int
```

Ahora muchas veces **no** queremos convertir:

```
nombre = input('Cómo te llamas? ') # se queda como str
```

Casting: `int('23515')` funciona, porque el string contiene sólo dígitos.

`int('veintitrés mil')` produce un error: Python no interpreta el texto, sólo reconoce dígitos.

Largo de un string: len

len(s) retorna la **cantidad de caracteres** de s.

```
frase = 'Sólo vuela'  
print(len(frase))    # 10  
print(len(''))       # 0  
print(len(' '))      # 1
```

Largo de un string: len

len(s) retorna la **cantidad de caracteres** de s.

```
frase = 'Sólo vuela'  
print(len(frase))    # 10  
print(len(''))       # 0  
print(len(' '))      # 1
```

Los **espacios también son caracteres** y cuentan para el largo.

Largo de un string: len

len(s) retorna la **cantidad de caracteres** de s.

```
frase = 'Sólo vuela'  
print(len(frase))    # 10  
print(len(''))       # 0  
print(len(' '))      # 1
```

Los **espacios también son caracteres** y cuentan para el largo.

len es una **función**: Recibe el string entre paréntesis.

Índices

Cada carácter tiene una **posición** (índice). Se accede con corchetes [].

P	y	t	h	o	n	i	a
0	1	2	3	4	5	6	7
-8	-7	-6	-5	-4	-3	-2	-1

Índices

Cada carácter tiene una **posición** (índice). Se accede con corchetes [].

P	y	t	h	o	n	i	a
0	1	2	3	4	5	6	7
-8	-7	-6	-5	-4	-3	-2	-1

```
s = 'Pythonia'
print(s[0])      # P
print(s[3])      # h
print(s[-5])     # h
print(len(s))    # 8
print(s[len(s)-1]) # a
```

Índices

Cada carácter tiene una **posición** (índice). Se accede con corchetes [].

P	y	t	h	o	n	i	a
0	1	2	3	4	5	6	7
-8	-7	-6	-5	-4	-3	-2	-1

```
s = 'Pythonia'
print(s[0])      # P
print(s[3])      # h
print(s[-5])     # h
print(len(s))    # 8
print(s[len(s)-1]) # a
```

El primer índice es 0, no 1.

El **último** índice válido es siempre $\text{len}(s) - 1$.

Índices negativos

Los índices negativos cuentan **desde la derecha**: -1 es el último carácter.

```
s = 'Pythonia'
```

```
print(s[-1])    # a
```

```
print(s[-2])    # i
```

```
print(s[-8])    # P
```

Índices negativos

Los índices negativos cuentan **desde la derecha**: -1 es el último carácter.

```
s = 'Pythonia'
print(s[-1])    # a
print(s[-2])    # i
print(s[-8])    # P
```

Útil para recuperar caracteres del final **sin preguntarnos por el largo**:

patente[-1] es la última letra.

Equivale a patente[len(patente)-1].

Índices negativos

Los índices negativos cuentan **desde la derecha**: -1 es el último carácter.

```
s = 'Pythonia'
```

```
print(s[-1])    # a
```

```
print(s[-2])    # i
```

```
print(s[-8])    # P
```

Útil para recuperar caracteres del final **sin preguntarnos por el largo**:

patente[-1] es la última letra.

Equivale a patente[len(patente)-1].

Si el índice no existe, el programa se cae:

```
print(s[8])      # IndexError: string index out of range
```

Inmutabilidad

Un string **no se puede modificar** una vez creado: los caracteres se pueden **leer**, pero no **escribir**.

```
nombre = 'Bodoque'
print(nombre[3])      # 'o'      <- leer: sí
nombre[3] = 'A'       # TypeError: 'str' object does
                      # not support item assignment
```

Inmutabilidad

Un string **no se puede modificar** una vez creado: los caracteres se pueden **leer**, pero no **escribir**.

```
nombre = 'Bodoque'
print(nombre[3])      # 'o'      <- leer: sí
nombre[3] = 'A'       # TypeError: 'str' object does
                      # not support item assignment
```

Lo que sí se puede es **reasignar la variable completa**:

```
nombre = 'Tulio'      # otra asignación, sin problema
```

Cambiar la variable no es lo mismo que cambiar el string.

Inmutabilidad

Un string **no se puede modificar** una vez creado: los caracteres se pueden **leer**, pero no **escribir**.

```
nombre = 'Bodoque'
print(nombre[3])      # 'o'      <- leer: sí
nombre[3] = 'A'       # TypeError: 'str' object does
                      # not support item assignment
```

Lo que sí se puede es **reasignar la variable completa**:

```
nombre = 'Tulio'      # otra asignación, sin problema
```

Cambiar la variable no es lo mismo que cambiar el string.

Toda “modificación” de un string es en realidad la **construcción de uno nuevo**.

Operadores sobre strings

Concatenar y repetir

+ **concatena** dos strings y crea uno nuevo.

```
a = 'Hola ' + 'mundo'
# 'Hola mundo'

nombre = 'Ada'
print('Hola ' + nombre)
```

* **repite** un string una cantidad de veces.

```
linea = 3 * 'Hola '
# 'Hola Hola Hola '

print('-' * 20)
```

Concatenar y repetir

+ **concatena** dos strings y crea uno nuevo.

```
a = 'Hola ' + 'mundo'
# 'Hola mundo'

nombre = 'Ada'
print('Hola ' + nombre)
```

* **repite** un string una cantidad de veces.

```
linea = 3 * 'Hola '
# 'Hola Hola Hola '

print('-' * 20)
```

+ sólo funciona entre strings:

```
print('Edad: ' + 25)      # TypeError
print('Edad: ' + str(25)) # ok, o bien: print('Edad:', 25)
```

El acumulador de strings

El **neutro** de la concatenación es el string vacío. Es el equivalente al $\text{suma} = 0$ de la unidad anterior.

```
acum = ''           # inicialización (antes del ciclo)
acum += 'a'         # 'a'
acum += 'b'         # 'ab'
```

El acumulador de strings

El **neutro** de la concatenación es el string vacío. Es el equivalente al $\text{suma} = 0$ de la unidad anterior.

```
acum = ''           # inicialización (antes del ciclo)
acum += 'a'         # 'a'
acum += 'b'         # 'ab'
```

Operación	Neutro	Acumulador
suma	0	<code>suma += x</code>
multiplicación	1	<code>prod *= x</code>
concatenación	''	<code>acum += c</code>

El acumulador de strings

El **neutro** de la concatenación es el string vacío. Es el equivalente al $\text{suma} = 0$ de la unidad anterior.

```
acum = ''           # inicialización (antes del ciclo)
acum += 'a'         # 'a'
acum += 'b'         # 'ab'
```

Operación	Neutro	Acumulador
suma	0	<code>suma += x</code>
multiplicación	1	<code>prod *= x</code>
concatenación	''	<code>acum += c</code>

Este patrón aparece en muchos problemas.

Pertenencia: in y not in

`c in s` retorna True si el carácter **o el substring** `c` aparece dentro de `s`.

```
nombre = 'Perro'
print('e' in nombre)           # True
print('E' in nombre)           # False  (mayúscula distinta)
print('rro' in nombre)         # True    (busca substrings)
if 'a' not in nombre:
    print('sin a')
```


Pertenencia: in y not in

c in s retorna True si el carácter **o el substring** c aparece dentro de s.

```
nombre = 'Perro'
print('e' in nombre)           # True
print('E' in nombre)           # False   (mayúscula distinta)
print('rro' in nombre)         # True     (busca substrings)
if 'a' not in nombre:
    print('sin a')
```

Cuidado: in busca en **cualquier** posición, no palabras completas.

```
lista = 'Vaca Pollo Cerdo Repollo Zapallo'
print('pollo' in lista)        # True ... pero por 'Repollo'
```

Substrings: el operador rebanador

`s[a:b]` crea un **nuevo** string desde el índice `a` hasta **uno antes** de `b`.

```
texto = 'gato grande, negro y gordo'  
print(texto[5:11])      # 'grande'
```

Substrings: el operador rebanador

s[a:b] crea un **nuevo** string desde el índice a hasta **uno antes** de b.

```
texto = 'gato grande, negro y gordo'  
print(texto[5:11])      # 'grande'
```

g	a	t	o		g	r	a	n	d	e	,
0	1	2	3	4	5	6	7	8	9	10	11

Substrings: el operador rebanador

`s[a:b]` crea un **nuevo** string desde el índice `a` hasta **uno antes** de `b`.

```
texto = 'gato grande, negro y gordo'
print(texto[5:11])      # 'grande'
```

g	a	t	o		g	r	a	n	d	e	,
0	1	2	3	4	5	6	7	8	9	10	11

- El largo de `s[a:b]` es `b - a` (si los índices son válidos).
- El extremo derecho **no se incluye**, igual que en `while i < n`.

Rebanadas abiertas

`s[:b]` desde el inicio hasta `b-1`.

```
texto[:4]    # 'gato'
```

`s[a:]` desde `a` hasta el final.

```
texto[-5:]   # 'gordo'
```

Rebanadas abiertas

`s[:b]` desde el inicio hasta `b-1`.

```
texto[:4]    # 'gato'
```

`s[a:]` desde `a` hasta el final.

```
texto[-5:]   # 'gordo'
```

Con rebanadas **no hay error** de índice:

```
s = 'gato'
s[2:100]     # 'to'
s[10:20]     # ''
```

Con `[ ]` un índice inválido sí es error.

Rebanadas abiertas

`s[:b]` desde el inicio hasta `b-1`.

```
texto[:4]    # 'gato'
```

`s[a:]` desde `a` hasta el final.

```
texto[-5:]   # 'gordo'
```

Con rebanadas **no hay error** de índice:

```
s = 'gato'
s[2:100]    # 'to'
s[10:20]    # ''
```

Con `[ ]` un índice inválido sí es error.

Así se “modifica” un string: `s[:i] + c + s[i+1:]` construye uno nuevo, con el carácter `i` cambiado.

Ejercicio rápido

Suponga `texto = 'gato grande, negro y gordo'`. ¿Qué retorna cada expresión?

```
texto[4]  
len(texto[5:8])  
texto[:4]  
texto[-5:]  
texto[100]
```


Ejercicio rápido

Suponga `texto = 'gato grande, negro y gordo'`. ¿Qué retorna cada expresión?

```
texto[4]
```

' ' (un espacio)

```
len(texto[5:8])
```

3

```
texto[:4]
```

'gato'

```
texto[-5:]
```

'gordo'

```
texto[100]
```

`IndexError`

Código ASCII y orden

ord y chr

Cada carácter tiene asociado un número entero: su **código ASCII**.

```
print(ord('A'))    # 65  
print(ord('a'))    # 97  
print(ord('0'))    # 48  
print(chr(65))     # 'A'
```

ord y chr

Cada carácter tiene asociado un número entero: su **código ASCII**.

```
print(ord('A'))    # 65
print(ord('a'))    # 97
print(ord('0'))    # 48
print(chr(65))     # 'A'
```

' '	(espacio)	32
'0' ... '9'		48 – 57
'A' ... 'Z'		65 – 90
'a' ... 'z'		97 – 122

ord y chr

Cada carácter tiene asociado un número entero: su **código ASCII**.

```
print(ord('A'))    # 65
print(ord('a'))    # 97
print(ord('0'))    # 48
print(chr(65))     # 'A'
```

' '	(espacio)	32
'0' ... '9'		48 – 57
'A' ... 'Z'		65 – 90
'a' ... 'z'		97 – 122

Mayúsculas y minúsculas son caracteres distintos, porque tienen códigos distintos. Y **todas** las mayúsculas van antes que **todas** las minúsculas.

Comparación de strings

Los strings se comparan con ==, !=, <, <=, >, >=.

La comparación es **lexicográfica**: se avanza carácter a carácter hasta encontrar el primer par distinto, y ahí decide el código ASCII.

Comparación de strings

Los strings se comparan con ==, !=, <, <=, >, >=.

La comparación es **lexicográfica**: se avanza carácter a carácter hasta encontrar el primer par distinto, y ahí decide el código ASCII.

```
'perro' < 'gato'  
'perro' == 'Perro'  
'perro' < 'Perro'  
'casa' < 'casas'
```

Comparación de strings

Los strings se comparan con ==, !=, <, <=, >, >=.

La comparación es **lexicográfica**: se avanza carácter a carácter hasta encontrar el primer par distinto, y ahí decide el código ASCII.

```
'perro' < 'gato'  
'perro' == 'Perro'  
'perro' < 'Perro'  
'casa' < 'casas'
```

False ('p' 112 > 'g' 103)

False ('p' ≠ 'P')

False ('p' 112 > 'P' 80)

True (prefijo: gana el más corto)

Comparación de strings

Los strings se comparan con ==, !=, <, <=, >, >=.

La comparación es **lexicográfica**: se avanza carácter a carácter hasta encontrar el primer par distinto, y ahí decide el código ASCII.

```
'perro' < 'gato'  
'perro' == 'Perro'  
'perro' < 'Perro'  
'casa' < 'casas'
```

False ('p' 112 > 'g' 103)

False ('p' ≠ 'P')

False ('p' 112 > 'P' 80)

True (prefijo: gana el más corto)

Es el orden del diccionario, salvo por las mayúsculas y los acentos.

Comparación de strings: un uso práctico

Las fechas en formato `aaaa-mm-dd` se pueden comparar **como strings**:

```
f1 = '2014-10-01'  
f2 = '2015-10-01'  
print(f1 <= f2)      # True
```

Comparación de strings: un uso práctico

Las fechas en formato `aaaa-mm-dd` se pueden comparar **como strings**:

```
f1 = '2014-10-01'  
f2 = '2015-10-01'  
print(f1 <= f2)      # True
```

Funciona porque el formato tiene **largo fijo** y va de lo más significativo a lo menos significativo.

Comparación de strings: un uso práctico

Las fechas en formato `aaaa-mm-dd` se pueden comparar **como strings**:

```
f1 = '2014-10-01'  
f2 = '2015-10-01'  
print(f1 <= f2)      # True
```

Funciona porque el formato tiene **largo fijo** y va de lo más significativo a lo menos significativo.

No funcionaría con `'1/6/2001'` y `'10/6/2001'`: ahí `'1' < '10'`, luego `'/' < '0'`...

Métodos

upper y lower

Retornan un **nuevo** string convertido a mayúsculas o minúsculas. Los caracteres que no son letras se copian sin cambios.

```
nombre = 'Juan Carlos Bodoque'  
mayusculas = nombre.upper()    # 'JUAN CARLOS BODOQUE'  
minusculas = nombre.lower()    # 'juan carlos bodoque'
```

upper y lower

Retornan un **nuevo** string convertido a mayúsculas o minúsculas. Los caracteres que no son letras se copian sin cambios.

```
nombre = 'Juan Carlos Bodoque'  
mayusculas = nombre.upper()    # 'JUAN CARLOS BODOQUE'  
minusculas = nombre.lower()    # 'juan carlos bodoque'
```

Ojo con la inmutabilidad:

```
nombre.upper()  
print(nombre)    # 'Juan Carlos Bodoque'  <- no cambió
```

El resultado hay que **guardarlo**.

Comparar ignorando mayúsculas

Patrón muy útil: llevar ambos strings a la misma caja antes de comparar.

```
resp = input('Continuar? (si/no) ')  
if resp.lower() == 'si':  
    ...
```


Comparar ignorando mayúsculas

Patrón muy útil: llevar ambos strings a la misma caja antes de comparar.

```
resp = input('Continuar? (si/no) ')  
if resp.lower() == 'si':  
    ...
```

```
if 'usm' in tweet.lower():  
    print('menciona a la universidad')
```

Comparar ignorando mayúsculas

Patrón muy útil: llevar ambos strings a la misma caja antes de comparar.

```
resp = input('Continuar? (si/no) ')  
if resp.lower() == 'si':  
    ...
```

```
if 'usm' in tweet.lower():  
    print('menciona a la universidad')
```

Así SI, Si, sI y si se aceptan por igual.

Operadores, funciones y métodos

Tres formas distintas de escribir una operación. Es importante distinguirlas:

Tipo	Se escribe	Ejemplos
operador	símbolo entre operandos	+ * in []
función	nombre(dato)	len(s) ord(c) chr(n)
método	dato.nombre()	s.upper() s.lower()

Operadores, funciones y métodos

Tres formas distintas de escribir una operación. Es importante distinguirlas:

Tipo	Se escribe	Ejemplos
operador	símbolo entre operandos	+ * in []
función	nombre(dato)	len(s) ord(c) chr(n)
método	dato.nombre()	s.upper() s.lower()

- Un **método** siempre se aplica *sobre* un dato: primero el string, luego un punto, luego el nombre.
- **upper(s)** no existe y **s.len()** tampoco.

Recorrer un string

Recorrido con while

Usamos un contador que va **de 0 a len(s)-1**: exactamente el patrón 1 de la unidad de ciclos.

```
s = input('Texto: ')\ni = 0\nwhile i < len(s):\n    print(s[i])\n    i += 1
```

Recorrido con while

Usamos un contador que va **de 0 a len(s)-1**: exactamente el patrón 1 de la unidad de ciclos.

```
s = input('Texto: ')
i = 0
while i < len(s):
    print(s[i])
    i += 1
```

- La condición es `i < len(s)`. **Con `<=` se produce un `IndexError`** en la última iteración.
- En cada iteración tenemos **el índice `i` y el carácter `s[i]`**.

Recorrido con for

Cuando sólo necesitamos los **caracteres** (y no sus posiciones), hay una forma más directa:

```
s = input('Texto: ')\nfor caracter in s:\n    print(caracter)
```


Recorrido con for

Cuando sólo necesitamos los **caracteres** (y no sus posiciones), hay una forma más directa:

```
s = input('Texto: ')\nfor caracter in s:\n    print(caracter)
```

- for toma **automáticamente** cada carácter del string, uno por uno, de izquierda a derecha.
- No hay inicialización, ni condición, ni actualización. **El ciclo nunca queda infinito.**
- La variable (caracter) se crea sola y cambia en cada iteración.

while o for?

for

- Recorrer **todos** los caracteres.
- No necesito saber la posición.

```
vocales = 0
for c in texto:
    if c in 'aeiou':
        vocales += 1
```

while

- Necesito el **índice** (`s[i+1]`, `s[i:i+3]`).
- Recorrer sólo **una parte**.
- Detenerme antes del final.

```
i = 0
while i < len(texto)-1:
    if texto[i] == texto[i+1]:
        print('repetido', i)
    i += 1
```

while o for?

for

- Recorrer **todos** los caracteres.
- No necesito saber la posición.

```
vocales = 0
for c in texto:
    if c in 'aeiou':
        vocales += 1
```

while

- Necesito el **índice** (s[i+1], s[i:i+3]).
- Recorrer sólo **una parte**.
- Detenerme antes del final.

```
i = 0
while i < len(texto)-1:
    if texto[i] == texto[i+1]:
        print('repetido', i)
    i += 1
```

Seguimos sin usar break ni continue.

Patrones sobre strings

Los patrones

#	Patrón	Idea
1	Contar	Contar caracteres que cumplen una condición
2	Filtrar / construir	Armar un string nuevo carácter a carácter
3	Buscar	Ver si aparece un carácter o un patrón
4	Mayor / menor	Comparar strings lexicográficamente
5	Separar en campos	Cortar por un separador (;, espacio, /)
6	Validar	Revisar que todos los caracteres sean admisibles

Los patrones

#	Patrón	Idea
1	Contar	Contar caracteres que cumplen una condición
2	Filtrar / construir	Armar un string nuevo carácter a carácter
3	Buscar	Ver si aparece un carácter o un patrón
4	Mayor / menor	Comparar strings lexicográficamente
5	Separar en campos	Cortar por un separador (;, espacio, /)
6	Validar	Revisar que todos los caracteres sean admisibles

Son los mismos patrones de la unidad de ciclos, aplicados a los caracteres.

1. Contar

Cuántas vocales tiene un texto:

```
texto = input('Texto: ')
vocales = 0
for c in texto:
    if c.lower() in 'aeiou':
        vocales += 1
print('Tiene', vocales, 'vocales')
```

1. Contar

Cuántas vocales tiene un texto:

```
texto = input('Texto: ')
vocales = 0
for c in texto:
    if c.lower() in 'aeiou':
        vocales += 1
print('Tiene', vocales, 'vocales')
```

`c.lower() in 'aeiou'` evita escribir `c=='a'` or `c=='e'` or ... y además cubre las mayúsculas.

2. Filtrar y construir

Copiar el texto, pero sin las vocales ni la puntuación:

```
texto = 'Esta es, una: frase corta'
resultado = ''
for c in texto:
    if c not in 'aeiouAEIOU' and c not in ' ,;.:' :
        resultado += c
print(resultado)      # 'stsnfrscrt'
```

2. Filtrar y construir

Copiar el texto, pero sin las vocales ni la puntuación:

```
texto = 'Esta es, una: frase corta'
resultado = ''
for c in texto:
    if c not in 'aeiouAEIOU' and c not in ' ,;.:' :
        resultado += c
print(resultado)      # 'stsnfrscrt'
```

El acumulador `resultado = ''` va **antes** del ciclo. Si queda dentro, se pierde todo en cada vuelta.

2. Filtrar y construir: invertir

Construir el string al revés:

```
texto = input('Texto: ')
invertido = ''
for c in texto:
    invertido = c + invertido
print(invertido)
```

2. Filtrar y construir: invertir

Construir el string al revés:

```
texto = input('Texto: ')
invertido = ''
for c in texto:
    invertido = c + invertido
print(invertido)
```

El truco: cada carácter nuevo se pone **adelante**, no atrás.

'a' → 'ba' → 'cba'

3. Buscar un patrón

Contar cuántas veces aparece 'GAT' dentro de una cadena de ADN:

```
texto = 'GATACCTAGATAAGCGATAG'
pat = 'GAT'
i = 0
cuenta = 0
while i <= len(texto) - len(pat):
    if texto[i:i+len(pat)] == pat:
        cuenta += 1
    i += 1
print(cuenta)      # 3
```

3. Buscar un patrón

Contar cuántas veces aparece 'GAT' dentro de una cadena de ADN:

```
texto = 'GATACCTAGATAAGCGATAG'
pat = 'GAT'
i = 0
cuenta = 0
while i <= len(texto) - len(pat):
    if texto[i:i+len(pat)] == pat:
        cuenta += 1
    i += 1
print(cuenta)      # 3
```

Aquí for no sirve: necesitamos el índice para tomar la rebanada texto[i:i+3].

4. Mayor / menor

La palabra alfabéticamente mayor de una lista de palabras leídas hasta el centinela:

```
palabra = input('Palabra (fin para terminar): ')
mayor = palabra
while palabra != 'fin':
    if palabra > mayor:
        mayor = palabra
    palabra = input('Palabra (fin para terminar): ')
print('La mayor es:', mayor)
```

4. Mayor / menor

La palabra alfabéticamente mayor de una lista de palabras leídas hasta el centinela:

```
palabra = input('Palabra (fin para terminar): ')
mayor = palabra
while palabra != 'fin':
    if palabra > mayor:
        mayor = palabra
    palabra = input('Palabra (fin para terminar): ')
print('La mayor es:', mayor)
```

Es el patrón de mayor/menor de la unidad anterior, cambiando números por strings.

5. Separar en campos

Un string trae varios datos separados por ;. Queremos procesarlos uno a uno.

```
linea = 'rut;rol;apellido1;apellido2;nombres'
campo = ''
i = 0
while i < len(linea):
    if linea[i] == ';':
        print(campo)
        campo = ''           # reiniciar para el siguiente
    else:
        campo += linea[i]
    i += 1
print(campo)                # el último campo
```

5. Separar en campos

Un string trae varios datos separados por ;. Queremos procesarlos uno a uno.

```
linea = 'rut;rol;apellido1;apellido2;nombres'
campo = ''
i = 0
while i < len(linea):
    if linea[i] == ';':
        print(campo)
        campo = ''           # reiniciar para el siguiente
    else:
        campo += linea[i]
    i += 1
print(campo)                # el último campo
```

El último campo no tiene separador después: se imprime al salir del ciclo.

5. Separar en campos: el truco del separador

Otra forma: agregar el separador al final y así **todos** los campos se cierran dentro del ciclo.

```
linea = 'Progra=78;Mate=83;Fisica=68' + ';'
campo = ''
for c in linea:
    if c == ';':
        print(campo)
        campo = ''
    else:
        campo += c
```

5. Separar en campos: el truco del separador

Otra forma: agregar el separador al final y así **todos** los campos se cierran dentro del ciclo.

```
linea = 'Progra=78;Mate=83;Fisica=68' + ';'
campo = ''
for c in linea:
    if c == ';':
        print(campo)
        campo = ''
    else:
        campo += c
```

Con esto se puede usar `for`, porque ya no necesitamos índices.

6. Validar

Una cadena de ADN es válida si sólo contiene A, C, G, T:

```
cadena = input('Cadena: ')
valida = True
for c in cadena:
    if c not in 'ACGT':
        valida = False
if valida:
    print('Cadena válida')
else:
    print('Cadena inválida')
```

6. Validar

Una cadena de ADN es válida si sólo contiene A, C, G, T:

```
cadena = input('Cadena: ')
valida = True
for c in cadena:
    if c not in 'ACGT':
        valida = False
if valida:
    print('Cadena válida')
else:
    print('Cadena inválida')
```

Patrón de bandera: valida empieza en True y sólo puede *empeorar*.

Ruteo

Ruteo: ¿qué hace este programa?

```
texto = input('Texto: ')
inicio = True
convertido = ''
for c in texto:
    if inicio:
        convertido += c.upper()
        inicio = False
    elif c == ' ':
        inicio = True
    else:
        convertido += c
print(convertido)
```

Rutee con 'en un lugar'.

c	inicio	convertido

Ruteo: ¿qué hace este programa?

```
texto = input('Texto: ')
inicio = True
convertido = ''
for c in texto:
    if inicio:
        convertido += c.upper()
        inicio = False
    elif c == ' ':
        inicio = True
    else:
        convertido += c
print(convertido)
```

Rutee con 'en un lugar'.

c	inicio	convertido

Pone en mayúscula la primera letra de cada palabra.

Ruteo: rebanadas

```
q = 'HBRU 93'  
s = q[1]  
q = 'XS-7122'  
c = q[3]  
s = s + c  
q = 'CM-4513'  
c = q[3]  
s = s + c  
print(s)
```

¿Qué imprime?

Ruteo: rebanadas

```
q = 'HBRU 93'  
s = q[1]  
q = 'XS-7122'  
c = q[3]  
s = s + c  
q = 'CM-4513'  
c = q[3]  
s = s + c  
print(s)
```

¿Qué imprime?

'B74'

q[1] de 'HBRU 93' es 'B'; q[3] de 'XS-7122' es '7'; q[3] de 'CM-4513' es '4'.

Errores frecuentes

Encuentre el error

Imprimir todos los caracteres de un string.

```
s = input('Texto: ')
i = 0
while i <= len(s):
    print(s[i])
    i += 1
```

Encuentre el error

Imprimir todos los caracteres de un string.

```
s = input('Texto: ')
i = 0
while i <= len(s):
    print(s[i])
    i += 1
```

IndexError: el último índice válido es $\text{len}(s)-1$, no $\text{len}(s)$.

```
while i < len(s):
```

Encuentre el error

Pasar un nombre a mayúsculas.

```
nombre = input('Nombre: ')\nnombre.upper()\nprint(nombre)
```

Encuentre el error

Pasar un nombre a mayúsculas.

```
nombre = input('Nombre: ')\nnombre.upper()\nprint(nombre)
```

No imprime nada distinto: los strings son inmutables, `upper()` **retorna** un string nuevo que aquí se pierde.

```
nombre = nombre.upper()
```


Encuentre el error

Contar las letras a de un texto.

```
texto = input('Texto: ')
for c in texto:
    cuenta = 0
    if c == 'a':
        cuenta += 1
print(cuenta)
```

Encuentre el error

Contar las letras a de un texto.

```
texto = input('Texto: ')\nfor c in texto:\n    cuenta = 0\n    if c == 'a':\n        cuenta += 1\nprint(cuenta)
```

cuenta = 0 quedó dentro del ciclo: se reinicia en cada carácter. El contador se inicializa **antes**.

Encuentre el error

Separar una frase en palabras.

```
frase = 'uno dos tres'  
palabra = ''  
for c in frase:  
    if c == ' ':  
        print(palabra)  
    else:  
        palabra += c
```

Encuentre el error

Separar una frase en palabras.

```
frase = 'uno dos tres'
palabra = ''
for c in frase:
    if c == ' ':
        print(palabra)
    else:
        palabra += c
```

- (1) Falta `palabra = ''` después del `print`: las palabras se van pegando.
- (2) Falta imprimir la **última** palabra, que no viene seguida de espacio.

Encuentre el error

Mostrar la edad ingresada.

```
edad = input('Edad: ')\nprint('El próximo año tendrás ' + edad + 1)
```

Encuentre el error

Mostrar la edad ingresada.

```
edad = input('Edad: ')\nprint('El próximo año tendrás ' + edad + 1)
```

TypeError: edad es un **string** y no se puede sumar 1.

```
edad = int(input('Edad: '))\nprint('El próximo año tendrás', edad + 1)
```

Lo que usamos en esta unidad

sí

- Literales con ' ' y " ".
- Índices positivos y negativos.
- Rebanadas [a:b], [:b], [a:].
- Operadores +, *, in, not in.
- Funciones len, ord, chr.
- Métodos upper(), lower().
- Comparación lexicográfica.
- Recorrido con while y con for.

NO

- Tercer argumento del rebanador (s[a:b:c]).
- Caracteres especiales (\n, \t) y strip().
- replace, index, find, split, join, format.
- break y continue.

Varios de éstos se estudian en la unidad de **texto y archivos**. Aquí procesamos los strings *a mano*, con ciclos.

Ejercicios

Ejercicio: letras que coinciden

Escriba un programa que construya un string con las letras que coinciden (**la misma letra en la misma posición**) en dos strings ingresados como entrada.

Texto 1: amorosos

Texto 2: amortiza

amor

Texto 1: conformidad

Texto 2: contorno

conor

Ojo: los strings pueden tener **distinto largo**.

Ejercicio: cadenas de ADN

Una cadena de ADN es válida si está compuesta únicamente por las bases Adenina (A), Citosina (C), Guanina (G) o Timina (T). La cadena viene en grupos de 4 letras separados por guiones.

(a) Escriba un programa que valide una cadena de ADN.

(b) Modifíquelo para que lea n cadenas, señale cuáles son válidas y, al final, indique cuántas fueron válidas y cuántas no.

Cadena: ACGT-TTGA-CCGA

Válida

Cadena: ACGT-TXGA

Inválida

Ejercicio: contraseña segura

Escriba un programa que determine si una contraseña es suficientemente segura. Lo es si tiene:

- al menos una letra **mayúscula**,
- al menos una **minúscula**,
- al menos un **dígito**,
- al menos un carácter de **puntuación** (. , ; :),
- y **largo mínimo 8**.

Ejercicio: contraseña segura

Escriba un programa que determine si una contraseña es suficientemente segura. Lo es si tiene:

- al menos una letra **mayúscula**,
- al menos una **minúscula**,
- al menos un **dígito**,
- al menos un carácter de **puntuación** (. , ; :),
- y **largo mínimo 8**.

Sugerencia: una bandera por cada condición, todas en False al comenzar.

Ejercicio: fechas mágicas

Una fecha es **mágica** si el día multiplicado por el mes es igual a los últimos dos dígitos del año. Por ejemplo, el 10 de junio de 1960: $10 \times 6 = 60$.

La fecha es un string en formato "mes día, año": un espacio separa mes y día, y una coma y un espacio separan el año.

Fecha: Junio 10, 1960

Es mágica

Fecha: Junio 11, 1960

No es mágica

Necesitará separar el string en sus partes y convertir a `int` lo que corresponda.

Ejercicio: tolerancia

Escriba un programa que lea dos strings y un nivel de **tolerancia** (entero no negativo). El programa debe indicar si los strings son iguales ignorando diferencias hasta la cantidad de tolerancia indicada.

```
Texto 1: perro  
Texto 2: perXo  
Tolerancia: 1  
Son iguales
```

```
Texto 1: perro  
Texto 2: perXo  
Tolerancia: 0  
Son distintos
```

Ejercicios

- (a) Escriba un programa que indique la **cantidad de veces** que un string aparece dentro de otro. Puede suponer que el primero tiene menor longitud que el segundo.
- (b) Escriba un programa que encuentre la **palabra de mayor longitud** dentro de un texto cuyas palabras se separan por un único espacio (sin espacio al final).
- (c) Escriba un programa que lea un texto y una palabra, y construya otro string igual al original pero **sin ninguna ocurrencia** de esa palabra.

Ejercicio: promedio de calificaciones

Dado un string con el siguiente formato, pero del que **desconocemos** la cantidad de asignaturas:

```
"Progra=78;Mate=83;Fisica=68;Quimica=65"
```

Escriba un programa que lea el string como entrada, calcule el **promedio** de las calificaciones e indique además la **asignatura con mejor nota**. En caso de empate puede mostrar cualquiera.

```
Promedio: 73.5
```

```
Mejor: Mate
```


Desafío: números romanos

Escriba un programa que lea un string con un número romano válido e imprima su valor decimal.

I	V	X	L	C	D	M
1	5	10	50	100	500	1000

Número romano: CLXIII

163

Número romano: CXLIX

149

Reglas:

- Si a la derecha de una cifra se escribe otra **igual o menor**, su valor se **suma**. Ej: XX = 20, XV = 15.
- Si a la izquierda de una cifra hay otra **menor**, esa se **resta**. Ej: IX = 9, CM = 900.
- Nunca se repite una misma letra más de tres veces seguidas.

Idea: compare cada carácter con el siguiente.